

Programación para aplicaciones de tiempo real

- Conceptos básicos sobre computadores y soporte físico
- Lenguajes de programación y sistemas operativos
- Conceptos de lenguaje C
- Normas POSIX para tiempo real

Definición de computador

- **Computador:**
 - “Máquina **programable** de **propósito general** que procesa datos”
 - El computador puede verse como una superposición de “máquinas virtuales”
 - Cada m.v. corresponde a un nivel de detalle, con su lenguaje y estructura

Niveles de máquina

- Diferentes niveles de detalle o abstracción
- Cada nivel tiene un lenguaje y una estructura

Máquina Simbólica
Máquina Operativa
Máquina Convencional
Micromáquina
Circuitos Lógicos
Circuitos Físicos
Dispositivos

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

3

Niveles de máquina

- Máquina convencional (puede ser la “real”)
 - Lenguaje: lenguaje de máquina
 - Estructura: Modelo de Von Neumann
- Máquina operativa
 - Máquina convencional + sistema operativo
 - Lenguaje: Lenguaje de máquina + servicios s.o.
 - Estructura: Módulos del s.o.
- Máquina simbólica
 - Lenguaje y estructura: Lenguaje de alto nivel (C, Ada95) y modelo de máquina que implica

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

4

Nivel de Máquina Convencional

- **Lenguaje: Lenguaje de máquina**
 - Codificación binaria de programa y datos
 - Programa almacenado en memoria
 - Instrucciones muy simples:
Copiar datos, sumar, multiplicar...
- **Estructura: Arquitectura de Von Neumann**
 - Memoria principal (MP)
 - Unidad de control (UC)
 - Unidad Aritmético-lógica (UAL)
 - Unidad de Entrada/Salida (E/S)

Máquina Simbólica
Máquina Operativa
Máquina Convencional
Micromáquina
Circuitos Lógicos
Circuitos Físicos
Dispositivos

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

5

Arquitectura de Von Neumann

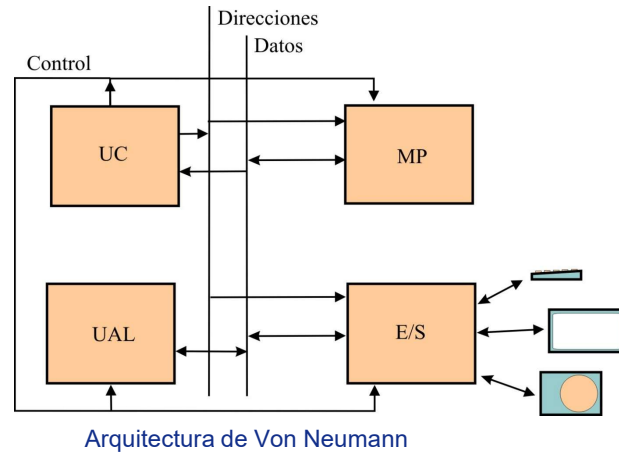
- **Memoria Principal**
 - Contenido: datos o instrucciones
 - Datos almacenados en palabras, cada una accesible por su **dirección**
 - Todo (código o datos) tiene una dirección
- **Unidad de Control**
 - Lee e interpreta instrucciones
- **Unidad Aritmético-lógica**
 - Dispone de circuitos lógicos para ejecutar instrucciones
- **Unidad de Entrada/Salida**
 - Comunica el computador con el exterior (todo menos MP, UC y UAL)
 - Circuitos de interfaz específicos

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

6

Arquitectura de Von Neumann



21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

7

Registros

- **Registros**
 - Contienen datos (como las palabras de MP)
 - Incluidos en determinados bloques: UAL, UC, E/S
 - Acceso diferente a MP (puede que no tengan dirección, o que esté en un espacio distinto...)
- **Diferentes tipos**
 - Aritméticos (operandos y resultados intermedios)
 - Direccionamiento (direcciones para MP)
 - Contador de programa (CP)
 - Puntero de pila (PP)
- **Registros de E/S**
 - Intercambio de datos con periféricos
 - Ejecución de comandos o lectura de estado de E/S

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

8

Registros

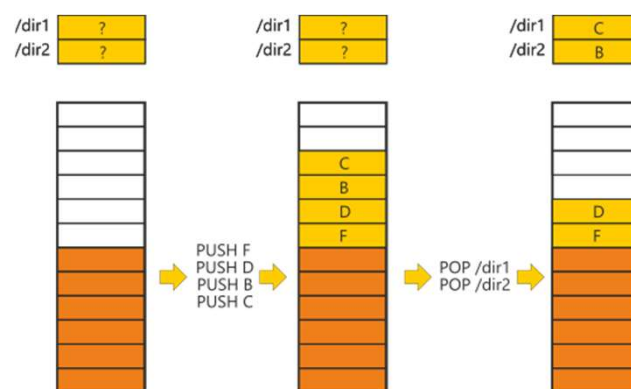
- **Contador de programa (CP):**
 - Define la instrucción actual
 - Normalmente se incrementa tras ejecutar la instrucción; también puede cambiarse (instrucciones de salto)
- **Puntero de pila (PP):**
 - Dedicado a implementar la **pila del sistema** (“stack”); define la “cima” de pila
 - Apunta a una palabra dentro de una zona de memoria reservada
- **Concepto de pila:**
 - Memoria LIFO
 - Dos operaciones básicas: Extraer datos (POP) y almacenar datos (PUSH)

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

9

Concepto de pila



Funcionamiento de una pila genérica (“stack”)

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

10

Pila del sistema

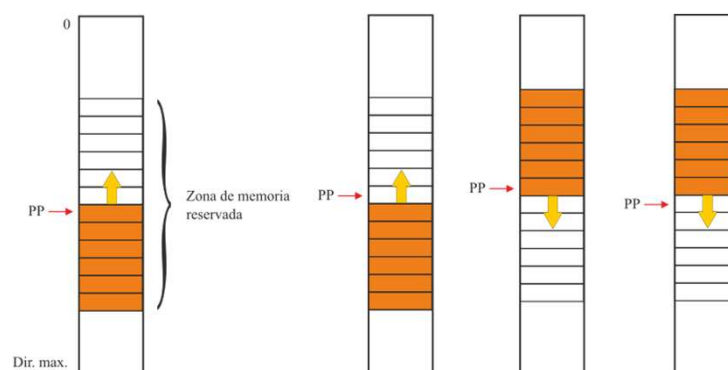
- Relacionada con las llamadas a función/subrutina
- Es una pila simulada en memoria:
 - Zona de memoria reservada
 - Los datos se van almacenando en posiciones sucesivas
 - Registro dedicado para definir la pila (PP)
 - Diferentes opciones de implementación:
 - Crecimiento hacia direcciones bajas o hacia direcciones altas
 - Puntero de pila que apunta a la última posición ocupada (cima) o a la primera libre
 - Pueden existir instrucciones de lenguaje máquina que **usan implícitamente el PP** (y por tanto la pila)
 - Almacenar datos
 - Extraer datos
 - Llamar a subrutina
 - Volver de subrutina

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

11

Pila del sistema



Existen diferentes opciones de implementación:

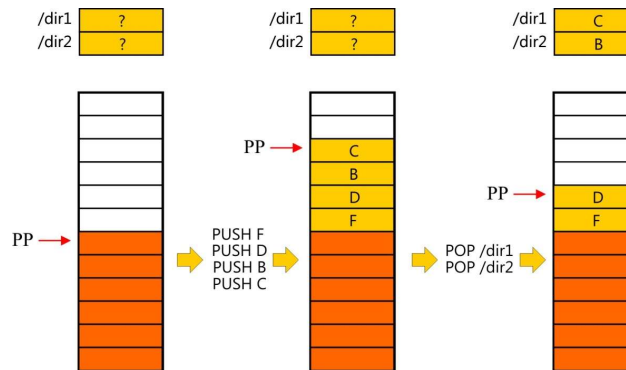
- Crecimiento hacia direcciones bajas o altas
- PP apunta a la última posición ocupada o a la primera posición libre

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

12

Pila del sistema



Pila que crece hacia direcciones bajas con PP apuntando a última posición ocupada

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

13

Pila del sistema

- Hay instrucciones que **utilizan implícitamente el PP** (y la pila...). Suponiendo que:
 - PP apunta a la cima (última posición ocupada)
 - La pila crece hacia direcciones bajas
- Almacenamiento y extracción de datos:
 - PUSH <valor>
Almacena <valor> en la pila:
 $PP = PP - 1$
 $M(PP) = \text{<valor>}$
 - POP /dir
Almacena cima de pila en /dir:
 $M(/dir) = M(PP)$
 $PP = PP + 1$
- Subrutinas – funciones:
 - CALL /dir
Almacena la DDR en la pila y salta a /dir:
 $PP = PP - 1$
 $M(PP) = DDR$
 $CP = /dir$
 - RET
Recupera la DDR de la pila y salta a DDR:
 $CP = M(PP)$
 $PP = PP + 1$

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

14

Funciones de la pila del sistema

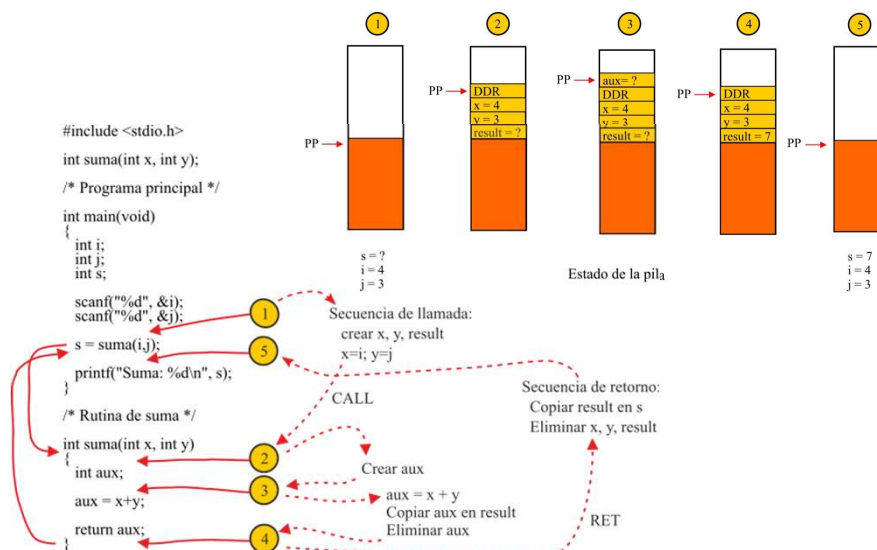
- Problemas a resolver en las llamadas a función:
 - Saltar a la función (trivial)
 - Volver al punto de llamada (dirección de retorno variable...)
 - Paso de argumentos y resultados
- Pila ("stack") de llamadas a función o pila del sistema:
 - Paso de la DDR a la función, y además...
 - Paso de argumentos y resultados
 - Creación de variables locales
 - Copia temporal de registros

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

15

Pila del sistema: llamada a función



21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

16

Pila del sistema: Llamada y retorno

- SP: “stack pointer” o puntero de pila (PP)
- En el punto de llamada:

(...)

* Secuencia de llamada

SUB #3, SP	* Crear x, y, result
MOVE /i, (SP)	* $x = i$ (o $M(SP) = i$)
MOVE /j, 1(SP)	* $y = j$ (o $M(SP+1) = j$)

* Saltar a subrutina

CALL /suma	* $PP = PP-1$; $M(PP)=CP$; $CP = /suma$
------------	---

* Secuencia de retorno

MOVE 2(SP), /s	* $s = result$ (o $s = M(SP+2)$)
ADD #3, SP	* Eliminar x, y,result

* Y la pila se queda ahora como estaba...

(...)

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

17

Pila del sistema

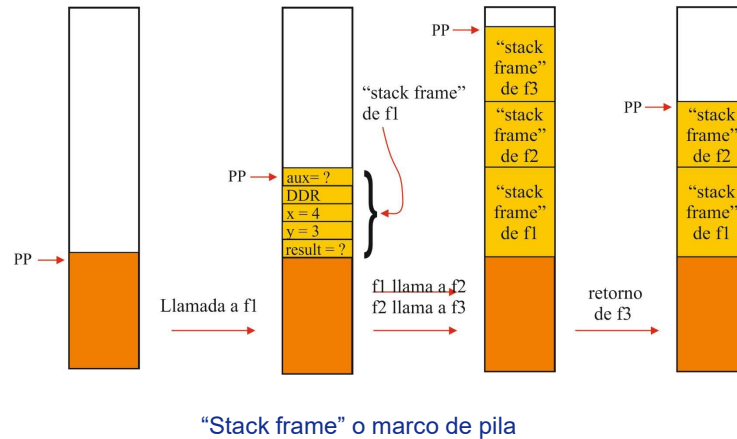
- Marco de pila:
 - Cada llamada a función crea una estructura de datos propia (“stack-frame” – marco de pila): Argumentos, dirección de retorno, variables locales...
 - Los marcos de pila de llamadas consecutivas se apilan
 - Al salir, los marcos de pila van desapareciendo por orden
 - Esta solución permite **recursividad** (una función puede llamarse a sí misma)
 - Si no usamos variables globales, cada invocación puede trabajar con datos, argumentos y DDR **distintos**

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

18

Pila del sistema



21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

19

Concurrencia y registros

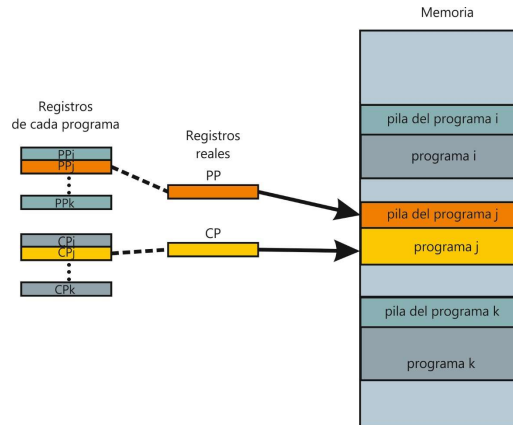
- **Concurrencia:** Ejecutar varias tareas en paralelo
- ¿Cómo puede simularse con un solo procesador?
 - Dedicar memoria separada a tareas diferentes
 - Multiplexar el acceso a la máquina convencional:
 - Conservar una **copia de los registros** relevantes (estado de la máquina) para las tareas que no se están ejecutando
 - **Cambio de contexto:** Salvar el estado actual y recuperar el estado de la tarea que pasa a utilizar la máquina
 - Especialmente evidente para
 - El contador de programa
 - El puntero de pila. Define la pila, y con ella la historia y el estado de todas las llamadas a función
 - Sin pilas separadas **no hay secuencias de llamadas a función separadas**

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

20

Concurrencia y registros



Múltiples programas comparten el computador (concurrencia)

21/09/2025

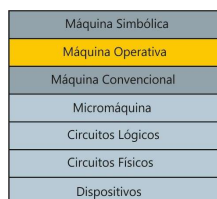
© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

21

Sistema operativo

• Máquina extendida

- Puede verse como una máquina virtual “sobre” la máquina convencional
- Realiza funciones complejas que el “hardware” real de la máquina no puede hacer:
 - Añade instrucciones a la máquina convencional (“llamadas” o servicios del S.O.)
 - Ofrece soluciones de alto nivel para problemas complejos (E/S, gestión de archivos)
 - Multitarea: crea varias “máquinas virtuales”



• Administrador de recursos

- Gestiona los recursos del computador: Tiempo de CPU, acceso a E/S, memoria
- Multiplexa (reparte) recursos en el espacio y en el tiempo

21/09/2025

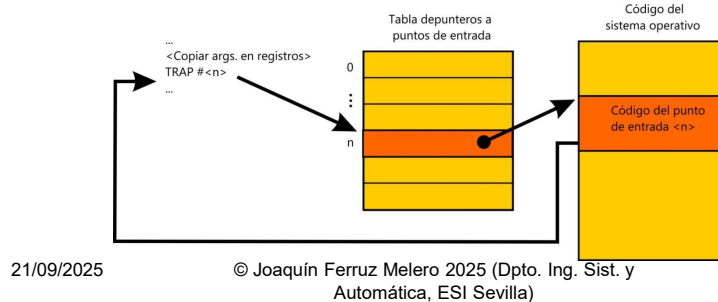
© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

22

Sistema operativo

• Implementación habitual

- Parte crítica (núcleo) asociada a un nivel de funcionamiento “privilegiado” del hardware distinto del nivel “normal” o “de usuario”
- Llamadas (“system calls”) implementadas mediante instrucciones de máquina especiales (TRAP, INT...)
- Las instrucciones especiales cambian el contexto y el nivel de privilegio controladamente:
 - No puedo invocar cualquier punto de entrada
 - La instrucción de llamada cambia el modo a privilegiado
 - La instrucción de retorno cambia de nuevo a modo de usuario



21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

23

Sistemas operativos y tiempo real

- **Varias opciones de implementación:**
 - Construir un sistema de propósito específico
 - Usar un lenguaje de alto nivel especializado y su sistema de soporte en tiempo de ejecución (por ejemplo ADA)
 - Usar los servicios de un S.O: mas un lenguaje convencional
- **Los sistemas operativos deben proporcionar:**
 - Soporte para concurrencia (multitarea)
 - Servicios de planificación (“scheduling”)
 - Servicios de temporización
 - Servicios de comunicación y sincronización
- **La mayor parte de los sistemas operativos parecen proporcionar lo necesario; ¿puede usarse cualquiera de ellos?**

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

24

Un poco de historia

- 1ª Generación (1945-55, válvulas): Sólo existe el nivel de máquina convencional
- 2ª Generación (1955-65, transistores):
 - Primeros lenguajes: FORTRAN
 - Proceso en tandas: s.o. elemental. Objetivo: **Eficiencia**
- 3ª Generación (1965-80, SSI, MSI)
 - Multiprogramación (OS-360 de IBM). Objetivo: **Eficiencia** (aprovechar tiempo de E/S)
 - Tiempo compartido (MULTICS, UNIX). Objetivo: **Reparto equitativo** de la potencia de cálculo entre usuarios
- 4ª Generación (1980-?, LSI)
 - Microprocesadores y computadores personales
 - Interconexión en red → s.o. en red y distribuidos

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

25

Sistemas operativos y tiempo real

- **Objetivos de diseño de los sistemas operativos convencionales:**
 - Eficiencia del uso del computador (maximizar computación por unidad de tiempo, o “throughput”)
 - Reparto equilibrado de la máquina entre aplicaciones y usuarios
- **Objetivos de los sistemas informáticos de tiempo real:**
 - Cumplir especificaciones temporales (un tiempo de respuesta suficientemente breve y **predecible**)
 - Se asigna la mayor prioridad a las tareas más urgentes
 - Las tareas no se tratan de manera equitativa
 - La eficiencia no es el fin principal
- **Conclusión: No todos los sistemas operativos son adecuados para tiempo real**

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

26

Sistemas operativos y tiempo real

- ¿Qué puede ser un problema en los s.o. convencionales?
 - No es realmente de un problema de funcionalidad
 - La estrategia de planificación puede ser inadecuada - las tareas más urgentes no tienen prioridad efectiva
 - Puede que tome decisiones a intervalo fijo ("tick") – las tareas más prioritarias tardan en entrar hasta un "tick"...
 - Es posible que las llamadas al s.o. no puedan interrumpirse, aunque el proceso que las llamó sea de baja prioridad
 - La prioridad efectiva es dinámica, no fija, y puede depender del comportamiento de la tarea (p.ej. sube si se bloquea; baja si no)
 - El tiempo de respuesta del propio sistema operativo puede ser impredecible
 - ¿Cuánto tarda en realizarse una llamada al s.o.? Puede que sea muy variable, y a veces muy grande...
 - Falta de adecuación para sistemas empotrados (necesidad elevada de memoria, dificultad para trabajar sin los periféricos habituales...)
- Algunos s.o. convencionales: Windows, UNIX, Linux

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

27

Sistemas operativos y tiempo real

- Sistemas pensados para aplicaciones de tiempo real ("RTOS"):
 - Planificación con expulsión efectiva de tareas menos prioritarias
 - Tiempo de respuesta de las llamadas predecible
 - Alta fiabilidad
 - Adaptabilidad a sistemas empotrados:
 - Modularidad: Puedo incorporar sólo la funcionalidad necesaria
 - Desarrollo en entorno cruzado: Desarrollo en computador y S.O. convencional ("host"); ejecución en sistema objetivo ("target") con el "R.T.O.S."; depuración remota
 - Bajos requerimientos mínimos de recursos (memoria, capacidad de cálculo, periféricos)



21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

28

Sistemas operativos y tiempo real

- Algunos “RTOS”:
 - QNX: Sistema comercial. Usado en vehículos y dispositivos médicos. Compatible POSIX
 - 255K vehículos de 45 marcas
 - Soluciones para multimedia, procesamiento de sensores, conducción automática...
 - Hypervisor para utilizar simultáneamente varios sistemas operativos (Linux, Android...)
 - VxWorks: Sistema comercial. Múltiples aplicaciones: aeroespaciales, automóviles, robótica industrial... Compatible POSIX
 - FreeRTOS: Código abierto. Compacto, soporte para microcontroladores. Compatibilidad parcial con POSIX
 - NuttX: Código abierto. Compacto, soporte para microcontroladores. Compatibilidad parcial con POSIX. Utilizado en el autopiloto Pixhawk

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

29

Algunos conceptos básicos sobre S.O.

- Proceso
- Interfaz de usuario
- Memoria virtual y direccionamiento paginado
- Archivos y directorios
- Llamadas al sistema

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

30

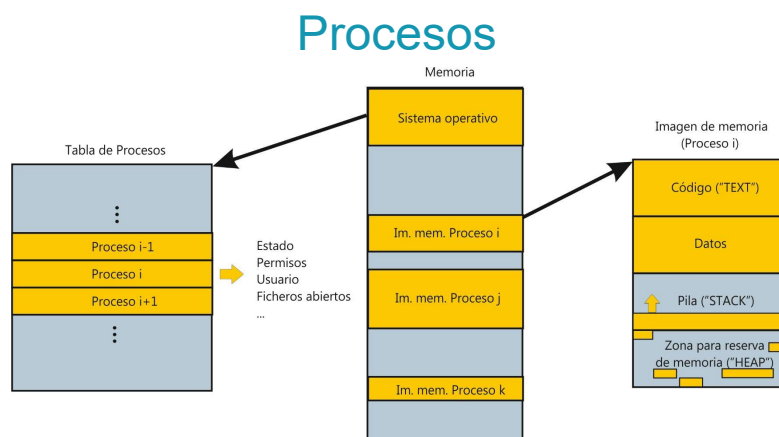
Procesos

- **Proceso**
 - Nombre típico para una unidad de concurrencia en la mayor parte de los sistemas operativos
 - No es un programa. Es un programa en ejecución
 - El mismo programa puede utilizarse en varios procesos
- **Dos componentes:**
 - Imagen de memoria
 - Creada a partir del programa ejecutable
 - Contiene instrucciones (programa) y datos
 - Entrada en la tabla de procesos
 - Estado de ejecución (p.e. copia de los registros)
 - Recursos utilizados (archivos, “sockets”...)
 - Datos administrativos (permisos, usuario...)

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

31



- **Intercambio de memoria ("memory swapping")**
 - Las imágenes de memoria no siempre están en memoria principal
 - Se vuelcan a disco imágenes de procesos inactivos
 - Permite mantener más procesos de los que caben en MP

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

32

Interfaz de usuario

- Interfaz de usuario:
 - Intérprete de comandos (“shell”) en modo texto
 - Interfaz gráfica de usuario (servidor de pantalla, gestor de ventanas)
- Hay que tener en cuenta que
 - Son el aspecto externo del s.o., **no deben identificarse con él**
 - Normalmente son procesos que utilizan los mismos servicios que los del usuario
 - Puede haber varias opciones para escoger

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

33

Direccionamiento virtual y paginado

- Algunos problemas a resolver en un s.o.:
 - Gestión de memoria (buscar lugar en memoria para un proceso)
 - Cuando entran y salen procesos, la memoria es ocupada y liberada según un patrón aleatorio
 - La memoria libre podría quedar demasiado dispersa para ser útil
 - Protección de datos (crear “barreras” entre procesos)
 - Reubicación (¡un proceso puede cambiar de dirección!)
 - En general el código tiene que almacenarse en las direcciones para la que se compiló; si no, no tiene por qué funcionar
 - Los procesos deben utilizar el espacio disponible; no pueden crearse para direcciones específicas
 - Si vuelven de disco, la memoria disponible puede haber cambiado
- Una medio muy extendido de resolver estos problemas:
Direccionamiento virtual y paginado

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

34

Direccionamiento virtual y paginado

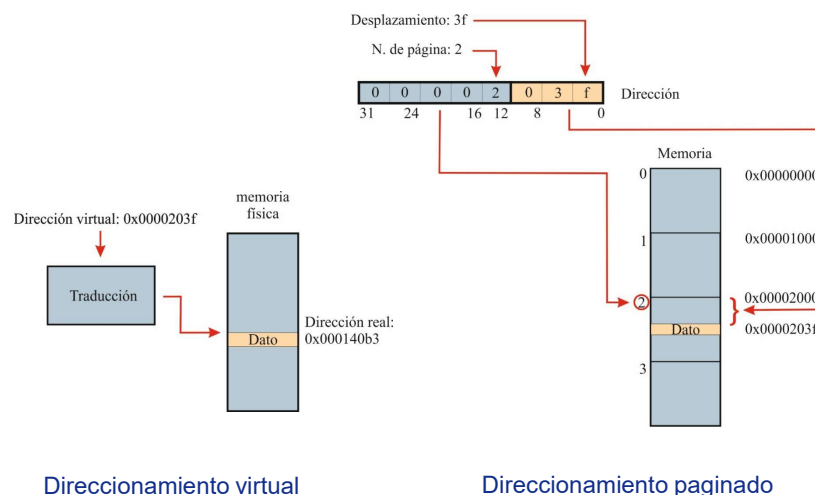
- Dos modos de direccionamiento combinados:
 - Direccionamiento virtual:
 - Cada proceso posee un **espacio de direcciones privado**
 - Se necesita traducción a direcciones reales
 - Direccionamiento paginado:
 - Memoria dividida en páginas de tamaño igual a una potencia de 2
 - Direcciones compuestas de número de página y desplazamiento
- Direccionamiento virtual y paginado:
 - La traducción de direcciones se hace utilizando una **tabla de páginas**
 - Cada página virtual se “mapea” como una unidad sobre una página real
 - Desplazamiento de dirección real: **El mismo** que el de la virtual
 - Número de página de dirección real: **El que indique la tabla de páginas**, usando el número de página virtual como índice

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

35

Direccionamiento virtual y paginado



21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

36

Direccionamiento virtual y paginado

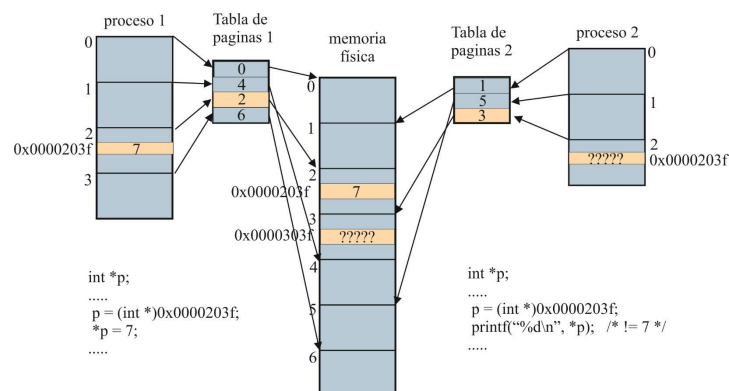
- **Tabla de páginas**
 - Una entrada para cada página virtual
 - Como mínimo: número de pagina real asociada
 - Otros datos: Bits de protección, indicador de página en disco
 - Excepción de fallo de página
 - Si la página no está en MP, el s.o. debe traerla de la memoria de intercambio
 - Es transparente al proceso (aunque la instrucción tarda más de lo normal en ejecutarse...)
- **Conclusiones:**
 - Solución de los problemas de gestión de memoria, protección y reubicación
 - Es necesaria circuitería especializada (MMU) para que sea viable

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

37

Direccionamiento virtual y paginado



- **En la figura:**
 - 20 bits de número de página
 - 12 bits de desplazamiento (página de 4096 octetos)
- **Una tabla de páginas por proceso**
- **El “mapeo” no se solapa: ¡No es inmediato compartir memoria!**

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

38

Direccionamiento virtual y paginado

- **Gestión de memoria:**
 - No hace falta que el espacio sea contiguo: Puede aprovecharse cualquier página libre
 - No hace falta que todas las páginas de un proceso estén en memoria a la vez
- **Protección:**
 - Asignación de memoria sin solapamiento: Espacios de direcciones de otros procesos inaccesibles
- **Reubicación:**
 - El proceso siempre “ve” las mismas direcciones virtuales (el código no tiene que cambiar; sólo la tabla de páginas)
 - Una página virtual puede residir en cualquier página física; basta configurar la tabla de páginas
- **Conclusiones:**
 - Solución de los problemas de gestión de memoria, protección y reubicación
 - Es necesaria circuitería especializada (MMU) para que sea viable

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

39

Sistema de archivos “tipo UNIX” y llamadas al sistema operativo

- **Archivos y directorios**
 - Un solo árbol de directorios (los discos individuales están ocultos por los puntos de montaje)
 - Todos los dispositivos de E/S están en el árbol como archivos
 - “Path name” y directorio de trabajo
 - Permisos tipo UNIX
 - Entrada y salida standard, salida de error
- **Llamadas al sistema**
 - Al nivel más bajo se ejecutan con instrucciones especiales
 - Habitualmente se utilizan funciones de biblioteca:
 - Interfaz adecuada al lenguaje
 - Operaciones a nivel de usuario
 - Una o más llamadas al sistema
 - Posibles bloqueos para sincronización (esperar datos)
 - Ejemplo: `cuenta = read(archivo, buffer, nbytes);`

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

40

Software de bajo nivel

- S.T.R: Dispositivos de E/S no standard. Puede ser necesario programar a bajo nivel
- Se prefiere hacerlo desde un lenguaje de alto nivel
- Requerimientos:
 - Acceso a registros de E/S
 - Tratamiento de interrupciones
 - Ejecución de instrucciones de lenguaje máquina

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

41

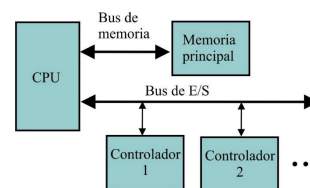
Software de bajo nivel

• Arquitectura de E/S

- Espacio de direcciones propio, con instrucciones especializadas:

in R1, <d. puerto E/S>

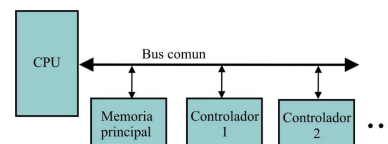
out <d. puerto E/S>, R1



- Espacio de direcciones compartido con memoria; instrucciones comunes:

move R1, <d. mem.>

move <d. mem.>, R1



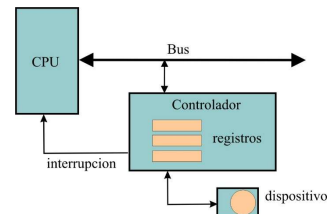
21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

42

Software de bajo nivel: Controlador

- **Dispositivo físico**
 - Compatible con interfaz del bus
 - Capaz de controlar uno o varios dispositivos de E/S
- **Comunicación con la CPU**
 - Registros: Transmisión de información
 - Control
 - Estado
 - Datos
 - Interrupciones: Sincronización mediante una señal dedicada



21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

43

Software de bajo nivel: Sincronización

- **Necesidad de sincronización**
 - Las acciones de E/S no son inmediatas
 - Normalmente el programa debe esperar al dispositivo (sincronización)
- **Métodos:**
 - Espera activa o consulta ("polling"):
 - El programa muestrea repetidamente el estado del controlador hasta detectar la condición (bit a 1, por ejemplo)
 - Simple pero poco eficiente: Ejecuta instrucciones inútilmente
 - Para ser admisible debe incluir espera si no se cumple la condición
 - Interrupciones:
 - El controlador envía una interrupción a la CPU
 - En respuesta se ejecuta una acción (rutina de servicio o manejador de interrupción)
 - Más eficiente pero con coste difícil de cuantificar
 - Acceso directo a memoria (DMA):
 - El propio controlador se encarga de almacenar los datos en memoria
 - Requiere interrupciones de sincronización, pero menos frecuentes
 - En principio más eficiente que las interrupciones
 - Coste difícil de cuantificar

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

44

Software de bajo nivel: Tratamiento de interrupciones

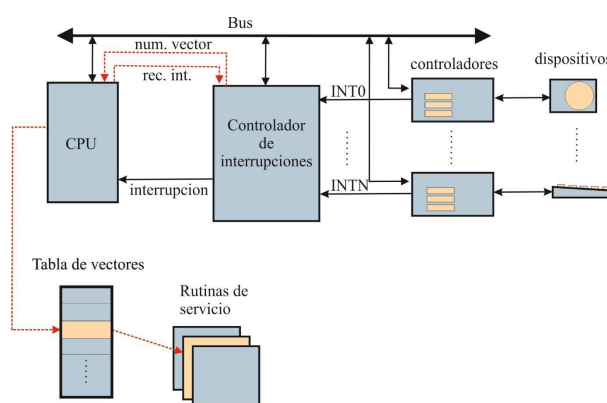
- Una interrupción activa una rutina de servicio
 - Es necesario almacenar el estado del procesador y recuperarlo luego
- Identificación de la causa:
 - Vector de interrupción: La CPU consulta automáticamente una tabla de “vectores” (punteros a función)
 - Estado: Una rutina de servicio “maestra” obtiene el número de interrupción de un registro de estado. La rutina de servicio específica arranca por programa
 - Consulta: Como el anterior, pero es necesario examinar cada dispositivo para identificar la causa
- Con frecuencia, el procedimiento es mixto:
 - Varias causas por señal de interrupción: Un vector por señal, y consulta para identificar la causa
 - Varias causas por dispositivo: Vector que identifica el dispositivo y consulta para identificar la causa

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

45

Software de bajo nivel: Tratamiento de interrupciones



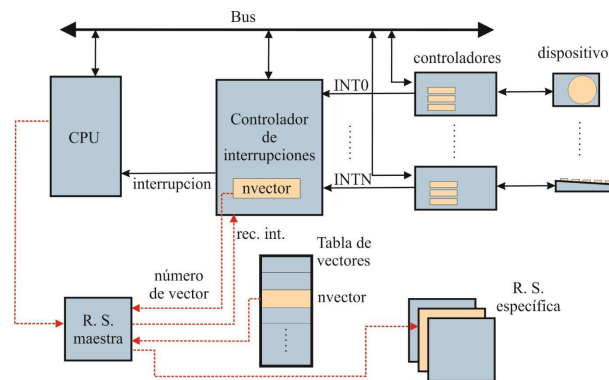
Consulta automática del vector de interrupción

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

46

Software de bajo nivel: Tratamiento de interrupciones



Identificación de la causa por registro de estado

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

47

Software de bajo nivel: Tratamiento de interrupciones

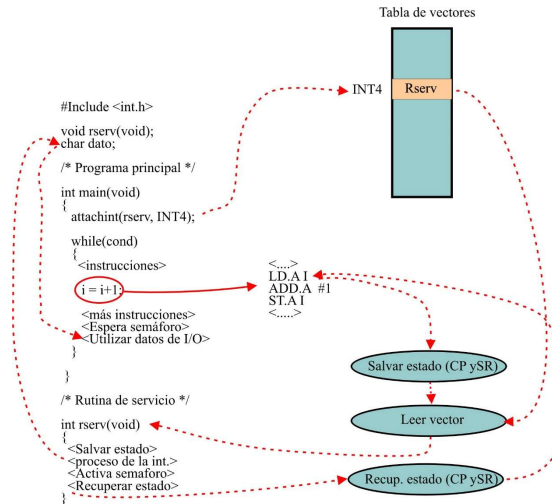
- **Prioridad:**
 - Pueden existir niveles de prioridad, para diferentes grados de “urgencia”
 - Ante varias causas pendientes, se atiende a la de mayor prioridad
 - Prioridad estática o dinámica
 - “Anidamiento” de interrupciones
- **Máscaras de interrupción**
 - Una señal de interrupción puede ignorarse temporalmente (señal enmascarada)
 - Varios niveles:
 - Habilidad global de interrupciones (CPU)
 - Máscaras de señales individuales (CPU/Contr. de Int.)
 - Máscaras de causas individuales (Bits de control en los controladores de dispositivos)
 - La prioridad puede entenderse como enmascaramiento selectivo

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

48

Software de bajo nivel: Interrupción vectorizada

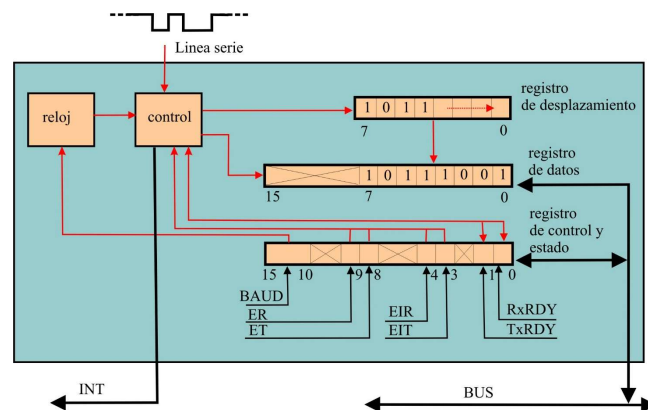


21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

49

Ejemplo de controlador: Línea serie (recepción)



- **Campos del RCE:**
 - RxRDY, TxRDY: Carácter recibido o transmitido.
 - EIR, EIT: Habilitar interrupción en recepción y transmisión
 - ER, ET: Habilitar recepción y transmisión
 - BAUD: Selección de velocidad de transmisión/recepción

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

50

Registros de entrada-salida

- Tienen dirección, aunque
 - Puede estar en un espacio separado al de la memoria principal (puede que se necesiten instrucciones especiales)
 - Es una dirección física, y el programa puede que trabaje con direcciones virtuales
- No se manejan del modo habitual:
 - Posible ancho parcial (no es una palabra completa)
 - Puede ser necesario tener en cuenta el ordenamiento de las direcciones de los octetos
 - Campos de ancho parcial con significado propio (bits, conjuntos de bits) !No se puede direccionar un bit!
 - Campos sin significado asignado ("basura"...)
 - Puede que permitan sólo escritura, o sólo lectura; y no siempre leeremos lo que hemos escrito

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

51

Software de bajo nivel y lenguaje

- Requerimientos para un lenguaje de alto nivel:
 - Representación de registros de E/S
 - Acceso a puertos de E/S o direcciones físicas (no siempre es simple: direccionamiento virtual y paginado...)
 - Programación de manejadores de interrupción (o rutinas de servicio)
 - Asociación del manejador a la interrupción
 - Almacenar y recuperar todos los registros
 - Posiblemente instrucción RET especial
 - Excepcionalmente, insertar código máquina
- En ADA:
 - Cláusulas de representación de registros, incluyendo detalles de bajo nivel
 - Asociación de manejadores a interrupciones, incluyendo sincronización con el resto del código (package Ada.Interrupts)
 - Inclusión de instrucciones de lenguaje máquina

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

52

Software de bajo nivel y lenguaje

- Software de bajo nivel en C
 - Acceso a registros fácil si el espacio de direcciones es compartido con memoria:
 - Punteros para acceder a registros
 - Operaciones lógicas bit a bit para manipular campos, o también estructuras con “campos de bits”
 - Problema: portabilidad
 - Manejadores de interrupción:
 - Funciones C sin parámetros ni resultado, posiblemente con declaración especial
 - Modificación directa de la tabla de vectores, usando funciones de biblioteca (no estandarizadas)
 - Es necesario resolver sincronización
 - Funcionalidad dependiente del entorno, no portable
 - Inclusión de código máquina:
 - Sentencias “asm” (catalogadas como “extensiones” de C).
 - Enlazar funciones en ensamblador, respetando las especificaciones de llamada y retorno del sistema concreto

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

53

Software de bajo nivel y lenguaje

- Software de bajo nivel en C

```
#define HAB_RX 0x200
#define HAB_TX 0x100
#define INT_RX 0x10
#define INT_TX 0x8
#define RRDY 0x1
#define TRDY 0x2
#define BAUD 0xA
#define DIR_RCE 0xffff1020
(...)
unsigned short int *reg_ce, prog;
/* Programar solo lectura con interrupción */

reg_ce = (unsigned short int *)DIR_RCE;
prog = (BAUD << 10) | HAB_RX | INT_RX;
*reg_ce = prog; /* Registro escrito */
(...)
```

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

54

Software de bajo nivel y lenguaje

- Software de bajo nivel en C

```
#define RRDY    0x1
#define DIR_RCE 0xffff1020
#define DIR_RD  0xffff1022
(...)
unsigned short int *reg_ce, short int *reg_dat, valor, dato;
reg_ce = (unsigned short int *)DIR_RCE;
reg_dat = (unsigned short int *)DIR_RD;

/* Bucle de espera activa */
do { valor = *reg_ce;
    if( (valor & RRDY) == 0) espera_1ms();
  } while( (valor & RRDY) == 0);
(...)
/* Leer dato */
dato = (*reg_dat)&0xff;
(...)
```

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y
Automática, ESI Sevilla)

55

Software de E/S

- Funciones del software de E/S:
 - Interfaz independiente del dispositivo
 - Nombres uniformes en el sistema de archivos
 - Tratamiento de errores
 - Sincronización del código con las actividades de E/S
 - Gestión de “buffers” intermedios
 - Control de acceso

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y
Automática, ESI Sevilla)

56

Software de E/S

- Niveles del software de E/S:
 - Software de E/S en nivel de usuario
 - Software de E/S en nivel de kernel, independiente del dispositivo
 - Controladores de dispositivos (“drivers”)
 - Manejadores de interrupción

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

57

Software de E/S

- Manejador de interrupciones en un s.o.
 - Pasos necesarios:
 - Guardar contexto del proceso actual y preparar contexto del manejador (pila, tabla de páginas...)
 - Operaciones comunes ligadas al hardware (reconocer interrupción, habilitar interrupciones...)
 - Procedimiento de servicio de interrupciones específico (lo que normalmente me dejan “tocar”...)
 - Escoger nuevo proceso y preparar nuevo contexto
 - Pasar control a nuevo proceso (no necesariamente aquél del que se ha partido)
 - Normalmente: operaciones sencillas y breves
 - Puede ser necesario desbloquear al controlador (“driver”)

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

58

Software de E/S

• Controlador (“driver”)

- Específico para el tipo de dispositivo
- Ofrece al resto del s.o. una determinada interfaz, ocultando las peculiaridades del dispositivo
- Funciones:
 - Desarrollar peticiones (leer o escribir N datos, por ejemplo) en forma de una secuencia de comandos
 - Se bloquea a la espera de interrupciones
 - Realiza tratamiento de errores
- Dificultades:
 - Peculiaridades en la interacción con el resto del s.o.
 - Puede necesitarse funcionamiento reentrante
 - Los recursos y condiciones de funcionamiento pueden cambiar

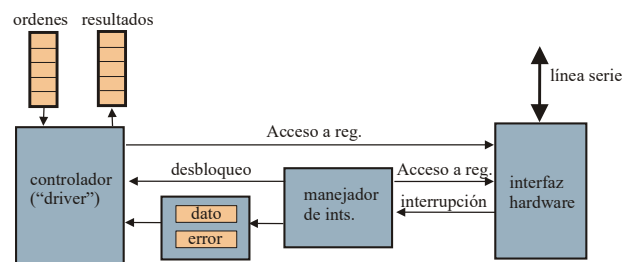
21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

59

Software de E/S

Controlador y manejador de interrupciones



Leer n datos en buffer

Habilitar lectura e int.
 Desde $i = 1$ hasta n
 Esperar interrupción
 Si error, devolver cod. error
 Copiar dato en buffer
 Fin desde
 Inhibir int. asociada
 Devolver resultado correcto

Atender int.

Salvar contexto
 Leer registro de error
 Si ha habido error,
 error = 1
 si no,
 dato = reg. de recepción
 fin si
 Desbloquear controlador
 Gestionar paso a nuevo proceso

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

60

Software de E/S

- Software de E/S independiente del dispositivo (nivel de kernel)
 - Realiza operaciones comunes y ofrece una interfaz única
 - Funciones:
 - Correspondencia entre nombres y dispositivos
 - Por ejemplo, el fichero /dev/disk0 representa al disco duro (o a su “driver”)
 - Sistema de archivos

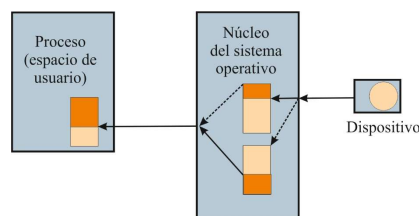
21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

61

Software de E/S

- Software de E/S independiente del dispositivo (nivel de kernel)
 - Funciones (cont.):
 - Gestión de buffers para transferencia de datos
 - Almacenamiento intermedio para datos (por ejemplo, de disco)
 - Caché de disco



21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

62

Software de E/S

- Software de E/S independiente del dispositivo (nivel de kernel)
 - Funciones (cont.):
 - Tratamiento de errores
 - Gestionar errores no específicos del dispositivo
 - Gestión de acceso a dispositivos
 - Impedir acceso simultáneo a determinados dispositivos (impresora, grabadora de discos ópticos)
 - Ocultar diferencias entre dispositivos
 - Unificar diferentes tamaños de bloque físico

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

63

Software de E/S

- Software de E/S en nivel de usuario
 - Bibliotecas de E/S y procesos del sistema
 - Funciones:
 - Ofrecer la interfaz requerida por el lenguaje de programación
 - Ejemplo: llamada con prototipo C para leer datos de archivo


```
ssize_t read(int f, void *buf, size_t nb);
```
 - Sincronización con operación de E/S
 - La E/S es asíncrona con el programa
 - Esperas internas para conseguir E/S **bloqueante**

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

64

Software de E/S

- Software de E/S en nivel de usuario

- Funciones (cont.):

- Formateo de datos

- Por ejemplo, para imprimir un entero debe codificarse en caracteres:

```
printf("¡ vale %d\n", i);
```

- Ejemplo de proceso de E/S: "Spooler"

- Proceso dedicado para gestiona el acceso a una impresora
 - Los procesos de usuario depositan ficheros a imprimir en una carpeta
 - El "spooler" los imprime por orden de llegada

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

65

Máquina simbólica

Máquina Simbólica
Máquina Operativa
Máquina Convencional
Micromáquina
Circuitos Lógicos
Circuitos Físicos
Dispositivos

- Máquina simbólica:

- Máquina virtual "sobre" la Máquina Operativa que "entiende" lenguajes de alto nivel
 - La máquina real no puede hacerlo
 - El programa de alto nivel es el **código fuente**

- Opciones de implementación:

- Interpretación: Un **intérprete** lee el código fuente como datos y lo ejecuta
 - Compilación: Un **compilador** traduce el código fuente a código máquina

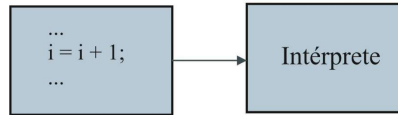
21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

66

Interpretación

Código fuente



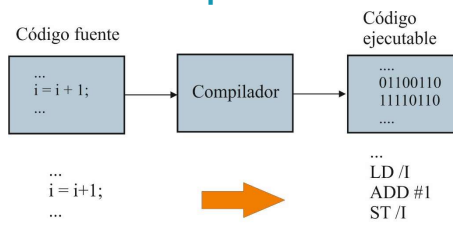
- El intérprete procesa el código fuente como datos
- El intérprete siempre se está ejecutando cuando se ejecuta el programa
- Muy portable pero habitualmente muy ineficiente si se compara con la compilación
- Ejemplos: archivos “m” de MATLAB, JAVA
 - JAVA se compila primero a “bytecode” (lenguaje intermedio)
 - La máquina virtual de JAVA, creada para un sistema específico, interpreta el “bytecode”

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

67

Compilación



- El compilador traduce el código fuente a un código ejecutable equivalente
- Una vez compilado, el código ejecutable puede ejecutarse tantas veces como sea necesario
- El compilador no ejecuta el código fuente; su papel acaba con la compilación
- Ejemplos: C, C++, ADA

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

68

Compilación separada

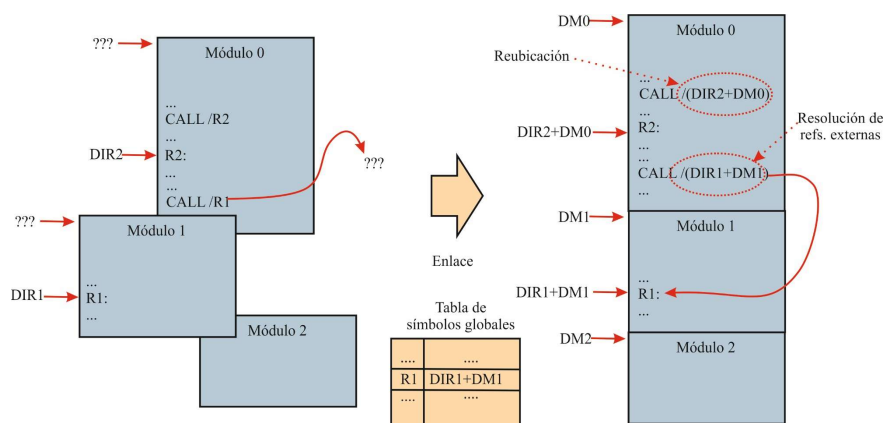
- Necesaria para modularidad
 - Permite construir el programa a partir de módulos
 - Cada módulo (fichero fuente) se compila separadamente
 - ¿Para qué direcciones? Difícilmente podrían ser definitivas...
- Se necesitan dos fases:
 - Compilación a objeto (sólo para los módulos que han cambiado):
 - Traduce código fuente para generar **código objeto reubicable**
 - No pueden asignarse direcciones definitivas
 - Las referencias externas (a elementos de otros módulos) están indefinidas (cambiarán si se recompilan...)
 - El código objeto puede adaptarse más tarde a direcciones arbitrarias
 - Edición de enlaces o "linking" (para todos los módulos)
 - Asignación definitiva de direcciones a cada módulo
 - Reubicación del código de todos los módulos – se adapta a las direcciones asignadas
 - Creación de una tabla de símbolos globales
 - Resolución de referencias externas

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

69

Compilación separada

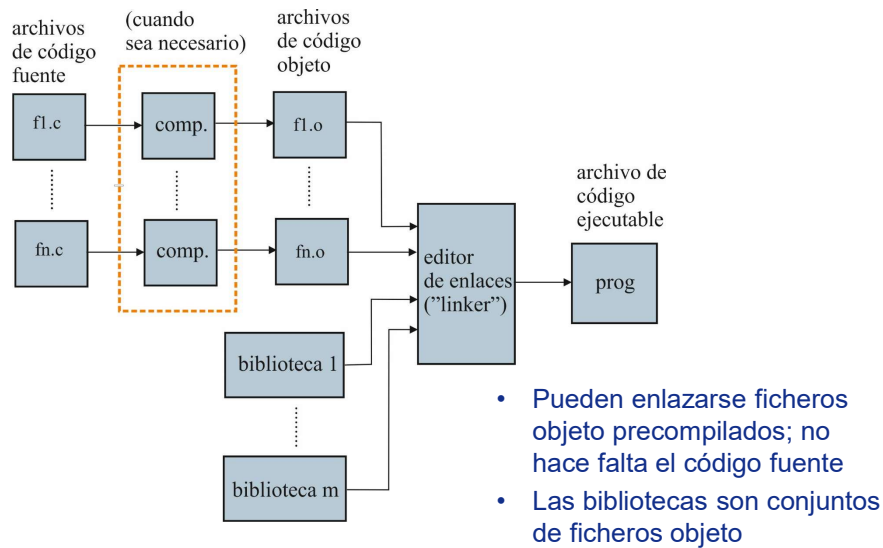


21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

70

Compilación separada



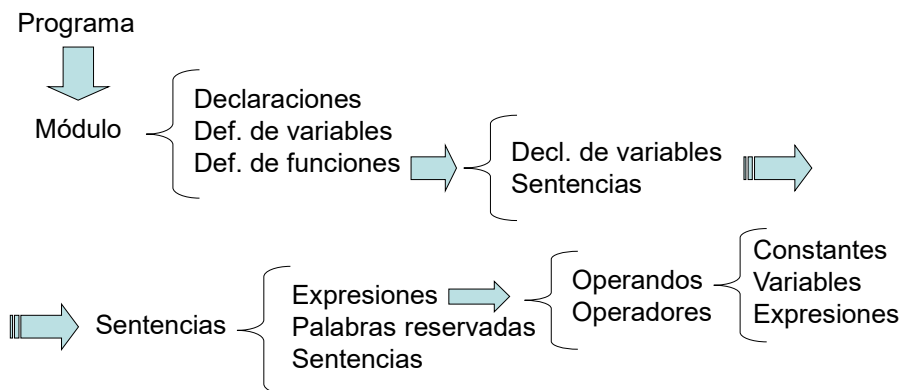
21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

71

Conceptos de lenguaje C

• Estructura del lenguaje



21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

72

Conceptos de lenguaje C

- Variables, constantes, operadores, expresiones
 - Tipos básicos
 - Visibilidad y tipo de almacenamiento
 - Funciones y argumentos
 - Vectores y matrices
 - Inicialización
 - Constantes
 - Sentencias y expresiones
 - Operadores
 - Precedencia y asociatividad
 - Conversión de tipo implícita
 - Orden de evaluación

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

73

Visibilidad

```

/* modulo1.c */
#include <stdio.h>
void f1(void);
int fext(int y);
int privada(int x);

/* Programa principal */
extern int externa1;
extern int externa3;

int main(void)
{
    int i;
    i = externa2;
    printf("externa1: %d\n", externa1);
    printf("externa3: %d\n", externa3);
    f1();
    f1();
    printf("fext: %d\n", fext(4));
    privada(i);
}

void f1(void)
{
    int aux;
    int externa3;
    static int estatica;
    externa3 = 7;
    aux = i;
    printf("externa1: %d\n", externa1);
    printf("externa3: %d\n", externa3);
    printf("estatica: %d\n", estatica);
    estatica = estatica + 1;
}
  
```

```

/* modulo2.c */
int externa1 = 27;
static int externa3 = 18;

int externa2;
int fext(int y)
{
    externa2 = y;
    return externa3;
}

static void privada(int x)
{
    externa3 = x;
}
  
```

Después de corregir errores:

```

> gcc -c modulo1.c
> gcc -c modulo2.c
> gcc -o prog.exe modulo1.o modulo2.o
> ./prog.exe
27
7
7
0
27
7
1
18
  
```

Simbolos globales de modulo1:

```
main
f1
```

Simbolos globales de modulo2:

```
externa1
externa2
fext
```

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

74

Operadores de C

SECTION 2.12

PRECEDENCE AND ORDER OF EVALUATION 53

TABLE 2-1. PRECEDENCE AND ASSOCIATIVITY OF OPERATORS

OPERATORS	ASSOCIATIVITY
() [] -> . *	left to right
! ~ ++ -- + - * & (type) sizeof	right to left
* / %	left to right
+ -	left to right
<< >>	left to right
< <= > >=	left to right
== !=	left to right
&	left to right
^	left to right
	left to right
&&	left to right
!!	left to right
? :	right to left
= += -= *= /= %= &= ^= = <<= >>=	right to left
,	left to right

Unary +, -, and ~ have higher precedence than the binary forms.

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

75

Conceptos de lenguaje C

- Sentencias de control
 - Programación estructurada
 - Bloques en secuencia
 - Sentencia compuesta
 - Operaciones condicionales
 - Sentencia **if-else**
 - Sentencia **switch**
 - Operaciones iterativas
 - Sentencia **for**
 - Sentencia **while**
 - Sentencia **do-while**
 - Saltos
 - Sentencia **break**
 - Sentencia **continue**
 - Sentencia **return**
 - Función **exit()**
 - Etiquetas y sentencia **goto**

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

76

Conceptos de lenguaje C

- Sentencia compuesta

```
{
    <sentencia 1>
    <sentencia 2>
    .....
}
```

- Sentencia **if-else**

```
if(<expresión>)
    <sentencia 1>
else
    <sentencia 2>
```

El "**else**" acompaña al "**if**" más cercano

- Sentencia **switch** (forma habitual)

```
switch(<expr. con resultado entero>)
{
    case <constante 1>:
        <sentencia 1>
        break;
    case <constante 2>:
        <sentencia 2>
        break;
    .....
    default:
        <sentencia n>
        break;
}
```

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

77

Conceptos de lenguaje C

- Sentencia **for**

```
for(<expresión inicial>; <expresión de mant.>; <expresión final (actualización)>)
    <sentencia>
```

- Sentencia **while**

```
while(<expresión (cond. de mantenimiento)>)
    <sentencia>
```

La sentencia **puede no llegar a ejecutarse** nunca

- Sentencia **do-while**

```
do
    <sentencia>
while(<expresión (cond. de mantenimiento)>)
```

La sentencia se ejecuta **al menos una vez**

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

78

Conceptos de lenguaje C

- Punteros y tablas (“arrays”)
 - Punteros y direcciones
 - Punteros y tablas
 - Aritmética de punteros
 - Inicialización de punteros
 - Cadenas de caracteres
 - Punteros a puntero
 - Punteros a función
 - Argumentos de la línea de comando

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

79

Punteros y tablas

- Punteros y direcciones
 - Declaración: `<tipo> *<identificador>;`

```
int i; float f;
int *pi;    /* Puntero a entero */
float *pf;  /* Puntero a float */
```
 - Un puntero es una variable que contiene la dirección de un tipo determinado (incluso podría ser la de un puntero...)
 - Operadores básicos: `&`, `*`

```
i = 7;
pi = &i;          /* Ahora pi “apunta” a i... */
printf("i=%d\n", *pi); /* ...y por tanto se imprimirá “7” */
pf = &f;          /* Ahora pf “apunta” a f */
pf = &i;          /* Oh, no debería hacer esto... (“warning”) */
pf = pi;          /* ni esto */
printf("i=%f\n", *pi); /* ...ni esto otro */
pf = (float *)pi; /* Esto se aceptaría, pero... */
```

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

80

Punteros y tablas

- Punteros y direcciones (cont.)

- Punteros genéricos: Carecen de tipo

```
void *pg; /* No se sabe a qué tipo apunta */
pg = pi; /* Puedo copiar cualquier puntero en uno genérico */
pf = pg; /* Y al revés (aunque quizás no tenga sentido...) */
i = *pg; /* No puedo usarlo para hacer referencia al dato */
i = *(int *)pg; /* Así sí... le estoy "dando" un tipo */
```

- Punteros y tablas

- El nombre de un "array" es un puntero constante al tipo de sus datos

```
int v[10]; int *pi; int i=7; /* v es un int * cte., igual que 7 es un int cte. */
pi = v; /* Correcto; v es un int * (cte.) y pi es un int * */
v[i] = 10; *(v+i) = 10; /* Expresiones equivalentes por definición */
pi[i] = 10; *(pi+i) = 10; /* Idem., y obtienen el mismo resultado */
pi = &i; /* Vale; pi es una variable */
v = &i; /* !No vale! !v es una cte.! (como hacer 7 = 3... */
```

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

81

Punteros y tablas

- Punteros y tablas (cont.)

- Para acceder al dato es esencial saber su tamaño
 - Por tanto, No puedo usar un puntero genérico con "[" – "]"
- Cuando paso una tabla a una función, en realidad paso un puntero
 - Estas declaraciones significan lo mismo:


```
int suma(int *p);
int suma(int p[]);
int suma(int p[3]); /* El 3 es irrelevante */
```
- En la función siempre puedo usar "p" como un puntero, y nadie controla el tamaño de la tabla...
- Nunca se pasa una copia de la tabla

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

82

Punteros y tablas

• Aritmética de direcciones

- Puedo sumar un puntero y un entero (como ya hemos visto...):

```
int *pi1; int i = 3; *(pi+i) = 7; *(pi+3) = 7; pi1 = pi + i; *pi1 = 7;
```

- Resultado: Una dirección desplazada **i** posiciones de **int** con respecto a aquella a la que apunta **pi** ("nos saltamos" **i** enteros)
- Si un entero ocupa 4 direcciones ("bytes"), **pi+i** es el contenido de **pi** más **i*4**

- Puedo restar dos punteros

- Resultado: Un entero (el número de datos entre ambos)

```
int v[10]; int *p1, *p2; int i;
```

```
p1 = &v[1]; p2 = &v[4];
```

```
i = p2 - p1; /* i valdrá 3... 3 posiciones de entero */
```

- También puedo comparar dos punteros, o incrementar/decrementar punteros
- No puedo multiplicar o sumar punteros (no tiene sentido...)
- No puedo copiar punteros a otro de tipo distinto sin "cast", salvo que uno sea **void ***

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

83

Punteros y tablas

• Inicialización de punteros

- Puedo inicializar punteros al declararlos:

```
int i;
```

```
int *pi = &i; /* &i es una dirección de entero constante */
```

```
int *pi1 = pi; /* Esto sólo puedo hacerlo si pi1 es automática */
```

- Puedo usar **malloc** para reservar memoria del "heap":

```
#include <stdlib.h>
```

```
/* void *malloc(size_t tam); void free(void *p); */
```

```
int *pi;
```

```
pi = malloc(10*sizeof(int)); /* Tabla de 10 enteros */
```

```
pi[3] = 7; *(pi+3) = 7;
```

```
free(pi); /* Libero memoria; ya no debo usar pi */
```

- A falta de una dirección lógica puedo inicializar con **NULL**, una dirección ilegal
 - Un acceso con **NULL** provoca una excepción; mejor eso que un puntero "descarriado"...

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

84

Punteros y tablas

- Punteros a puntero

- Un puntero puede apuntar a cualquier tipo, incluso a otro puntero:

```
int **pp; int *p; int i; int j;
p = &i; pp = &p;
**pp = 7;          /* 7 va a i */
*p = 7;            /* 7 va a i */
*pp = &j;          /* *pp (o sea, p) es un int *, y también &j */
**pp = 8;          /* Ahora 8 va a j, porque p ha cambiado */
```

- Tabla o "array" de punteros:

```
int *pt[10];          /* Tabla de 10 punteros a entero;
                      es como int *(p[10]) */
int (*pt1)[10];       /* Esto es otra cosa completamente distinta... */
int x; int z[5];
pt[3] = &x; pt[4] = z; /* Correcto */
*pt[4] = 18;          /* 18 va a z[0]; también valdría *(p[4]) */
pt = pt+1;            /* !no! !pt es una cte.! */
```

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

85

Punteros

```
#include <stdio.h>
void suma(int x, int y, int *res);
/* Programa principal */
int main(void)
{
    /* Declaraciones */
    int i, j, result;
    int *p;
    int v[4] = {0, 1, 2, 3};
    /* Sentencias */
    i = 7;
    p = &i;
    printf("i: %d, p: %p\n", i, p);
    *p = 18;
    printf("i: %d, p: %p\n", i, p);
    p = v;
    printf("v[1]: %d, p[1]: %d, *(p+1): %d, p: %p, p+1: %p\n",
          v[1], p[1], *(p+1), p, p+1);
    i = 4; j = 7;
    suma(i, j, &result);
    printf("i: %d, j: %d, result %d\n", i, j, result);
}
/* Rutina de suma */
void suma(int x, int y, int *p)
{
    *p = x + y;
}
```

Diagram illustrating memory addresses and values for variables:

Variable	Value	Address
i	7	0x00ff4000
j	7	0x00ff4002
result	7	0x00ff4004
p	0x00ff4000	0x00ff4006
v[0]	0	0x00ff400A
v[1]	1	0x00ff400C
v[2]	2	0x00ff400E
v[3]	3	0x00ff4010

Diagram illustrating memory addresses and values for variables:

Variable	Value	Address
i	18	0x00ff4000
j	7	0x00ff4002
result	7	0x00ff4004
p	0x00ff400A	0x00ff4006
v[0]	0	0x00ff400A
v[1]	1	0x00ff400C
v[2]	2	0x00ff400E
v[3]	3	0x00ff4010

Al ejecutar el programa:

```
> prog
i: 7, p: 00ff4000
i: 18, p: 00ff4000
v[1]: 1, p[1]: 1, *(p+1): 1, p: 00ff400A, p+1: 00ff400C
i: 4, j: 7, result: 11
```

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

86

Cadenas de caracteres

- Caracteres almacenados en posiciones sucesivas
- Cadena definida por una dirección y un terminador ('\0')
 - Un puntero a **char** define una cadena
 - Una cadena constante puede representar un conjunto de valores o un char *:


```
char *pc = "ABC"; /* "ABC" equivale a un char * constante */
char cad[10] = "ABC"; /* Ahora es una secuencia de datos... */
char cad1[] = "123"; /* Se reservan 4 octetos para la cadena */
```
- Funciones para cadenas de caracteres:
 - No puedo sumar o copiar cadenas directamente (es C, no C++...)

```
#include <string.h>
size_t strlen(char *cadena);
char *strcpy(char *destino, char *origen);
char *strcat(char *destino, char *origen);
int strcmp(char *cad1, char *cad2);
```

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

87

Cadenas de caracteres

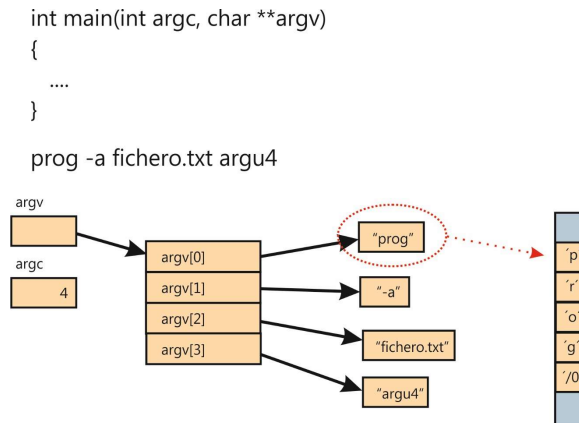
- Función **strlen**:
 - Devuelve el número de caracteres de la cadena, sin contar el terminador
- Función **strcpy**:
 - Copia la cadena origen en la cadena destino
 - Devuelve un puntero a la cadena destino
- Función **strcat**:
 - Añade la cadena origen al final de la cadena destino
 - Devuelve un puntero a la cadena destino
- Función **strcmp**:
 - Compara las cadenas 1 y 2
 - Devuelve 0 si son iguales, >0 si la cadena 1 debe ir detrás de la 2 y <0 si es al contrario

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

88

Argumentos de la línea de comando



21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

89

Argumentos de la línea de comando

- Los argumentos de l.c. son **cadenas de caracteres**
- Es necesario realizar una **conversión** para interpretarlos como otro tipo (**int**, **float**...)
- Funciones **scanf**, **fscanf**, y **sscanf**:
 - Función **scanf**: lee con formato de la entrada standard:
`int scanf(char *formato,...);`
 - Función **fscanf**: igual, pero lee de un archivo:
`int fscanf(FILE *fichero, char *formato,...);`
 - Función **sscanf**: igual, pero “lee” de una cadena de caracteres:
`int sscanf(char *input, char *formato,...);`
 - La función **sscanf** puede ser útil para convertir argumentos de la l.c.

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

90

Argumentos de la línea de comando

- Ejemplo:
 - Línea de comandos: prog a 7 b
 - Programa:


```
(...)
#include <stdio.h>
(...)
int main(int argc, char **argv) {
    int a1; /* !No podemos hacer a1 = argv[2]! */
    sscanf(argv[2], "%d", &a1); /* Suponemos que esto es posible */
    /* Ahora a1 vale 7 */
    (...)
}
```
 - ¿Cómo saber si la conversión se ha realizado?
 - Resultado de **sscanf** (o **scanf**, o **fscanf**): Número de campos convertidos, o **EOF** si error o fin de archivo

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

91

Conceptos de lenguaje C

- Estructuras
 - Definición y declaración
 - Inicialización
 - Acceso a elementos
 - Tablas de estructuras
 - Estructuras anidadas
 - Estructuras como argumentos de funciones
 - Estructuras con autoreferencia

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

92

Conceptos de lenguaje C

- Otros conceptos de C
 - Uniones
 - Definición de tipos
 - Tipos enumerados
 - Comandos del preprocesador
 - Campos de bits

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

93

Conceptos de lenguaje C

- Entrada-salida y ficheros
 - Conceptos básicos
 - Apertura y cierre de ficheros
(**fopen**, **fclose**)
 - Lectura y escritura de caracteres aislados
(**fgetc**, **fputc**, **getchar**, **putchar**)
 - Lectura y escritura de cadenas de caracteres
(**fgets**, **fputs**, **gets**, **puts**)
 - Lectura y escritura formateada
(**fprintf**, **fscanf**, **printf**, **scanf**)
 - Lectura y escritura de bloques
(**fread**, **fwrite**)
 - Acceso aleatorio
(**fseek**, **ftell**)
 - Detección de error y fin de archivo
(**ferror**, **feof**)

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

94

Normas POSIX para tiempo real

- Definidas por el IEEE (“Institute of Electrical and Electronics Engineers”), como otras:
 - IEEE 802.3, (Ethernet)
 - IEEE 802.11 (Wifi)...
- Estandarizadas por ANSI e ISO
- POSIX: “Portable Operating System interface”
 - Familia de normas que definen aspectos básicos de sistemas operativos: IEEE 1003.x
 - IEEE 1003.1 define las llamadas al sistema
- Accesibles en línea en versión .pdf (a través de la Biblioteca) o HTML

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

95

Normas POSIX para tiempo real

- Algunos componentes de la norma 1003.1:
 - POSIX 1003.1a (antes .1):
 - UNIX normalizado. Funciones básicas: Concurrencia, entrada/salida, temporización...
 - Se solapa con ANSI-C por definición
 - POSIX 1003.1b (antes .4):
 - Funciones llamadas de “de tiempo real”: Servicios más avanzados de planificación, temporización, comunicación...
 - Numerosos bloques opcionales
 - POSIX 1003.1c (antes .4a): Hilos y llamadas asociadas
- Hay varias ediciones, y un cierto número de componentes opcionales

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

96

Normas POSIX para tiempo real

- Comprobación de opciones en compilación
 - Macro `_POSIX_C_SOURCE`: normas que necesitamos
 - 199305L : Primera versión de POSIX 1.b
 - 199506L : Primera versión de POSIX 1.c
 - 200112L : Última versión que soporta QNX 6.3.2
 - 200809L : Última versión que soporta Ubuntu (20.04)
 - Las cabeceras ofrecerán esa funcionalidad, si está disponible (compilación condicional)
 - Macros para opciones y límites:
 - `_POSIX_REALTIME_SIGNALS`: Señales de tiempo real
 - `SIGQUEUE_MAX`: Número de señales que pueden encolarse
 - `_POSIX_SIGQUEUE_MAX`: Mínimo valor para `SIGQUEUE_MAX`
 - La compilación puede adaptarse, o fallar si no existe la funcionalidad requerida

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

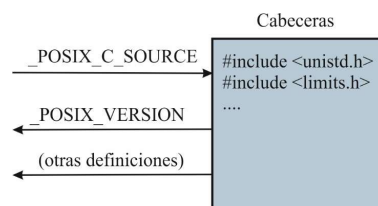
97

Normas POSIX para tiempo real

- Comprobación de opciones en compilación (cont.)
 - Un programa compatible POSIX debería definir `_POSIX_C_SOURCE` antes de las cabeceras:

```
#define _POSIX_C_SOURCE 199309L /* POSIX 1.b */
#include <unistd.h> /* Macros de existencia de opciones
                    (entre otras cosas...) */
#include <limits.h> /* Límites de parámetros */
#include <....
```

- `_POSIX_VERSION` indicará la última edición disponible



21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

98

Normas POSIX para tiempo real

- Comprobación de opciones en tiempo de ejecución:
 - Puede que las condiciones de funcionamiento hayan cambiado
 - Basada en funciones


```
(...)
#include <unistd.h>    /* Declaración de long sysconf(int name); */
result = sysconf(_SC_SEMAPHORES);    /* Si result = -1, no existen
                                      semáforos */

result = sysconf(_SC_SEM_VALUE_MAX); /* Devuelve valor max. de
                                      un semáforo */
```
 - El programa puede adaptarse, o bien fallar con un mensaje de error

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y
Automática, ESI Sevilla)

99

Normas POSIX para tiempo real

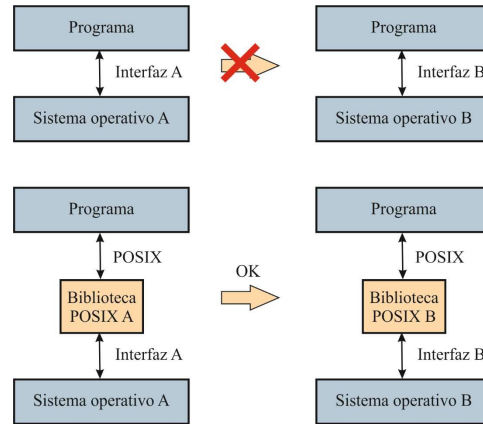
- Idea fundamental: Hacer posible la **portabilidad** entre diferentes sistemas operativos
- Soportadas por un número creciente de sistemas operativos: Linux, QNX, VxWorks...
- Habitualmente implementadas como una biblioteca que provee servicios POSIX a partir de los nativos
- Las normas POSIX especifican sobre todo requerimientos funcionales, **no de tiempos de respuesta**
- Soportar las normas POSIX **no implica que el sistema operativo sea apropiado para tiempo real**

21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y
Automática, ESI Sevilla)

100

Normas POSIX para tiempo real



21/09/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

101